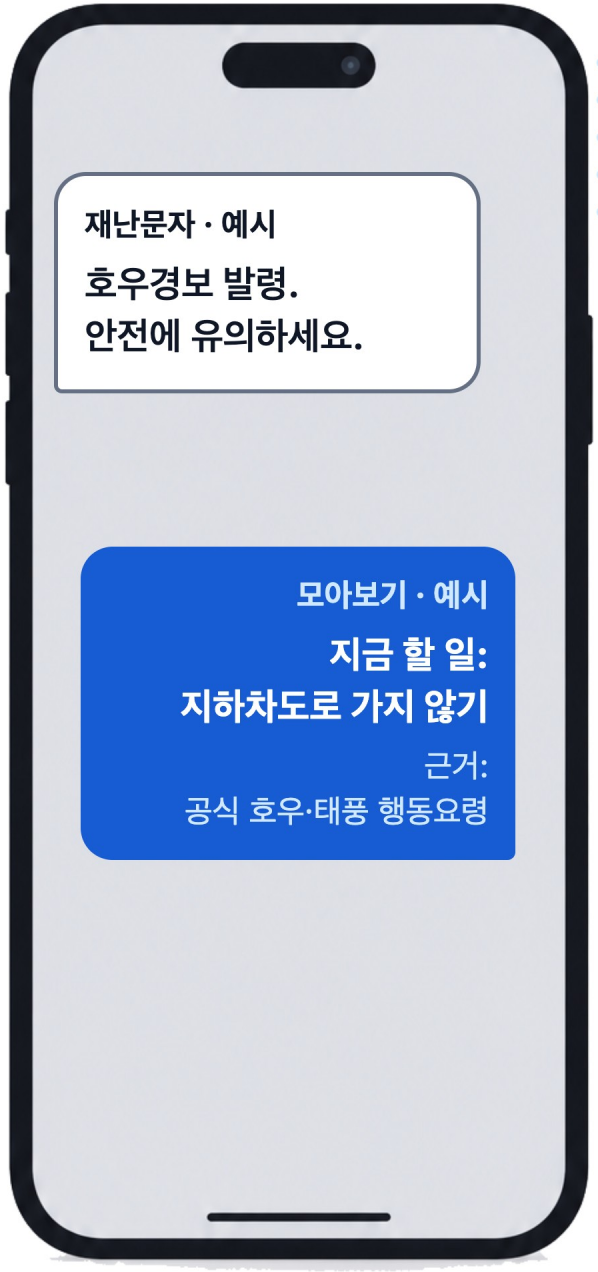
재난문자 모아보기 · 공공데이터 활용 AI 에이전트

문자는 무슨 일인지만 알려 준다.

우리는 지금 할 일을 알려 준다.

광주 4반 한세훈(G137)



문제

문자는 오는데, 무엇을 해야 할지 모른다

문자는 짧다

"호우경보 발령"

단계만 알려 주고, 지금 해야 할 행동은 없다.

정보가 흩어져 있다

특보·문자·행동요령·대피소가 API 8개 (공공 7종 + 지도), 발급처 3곳에 나뉘어 있다.

AI는 지어낼 수 있다

공식 근거 없이 답하면 그럴듯하지만 틀린 안내가 나온다.

20,259건

2023년 전국 재난문자 발송 수. 2019년까지는 연평균 414건이었다.

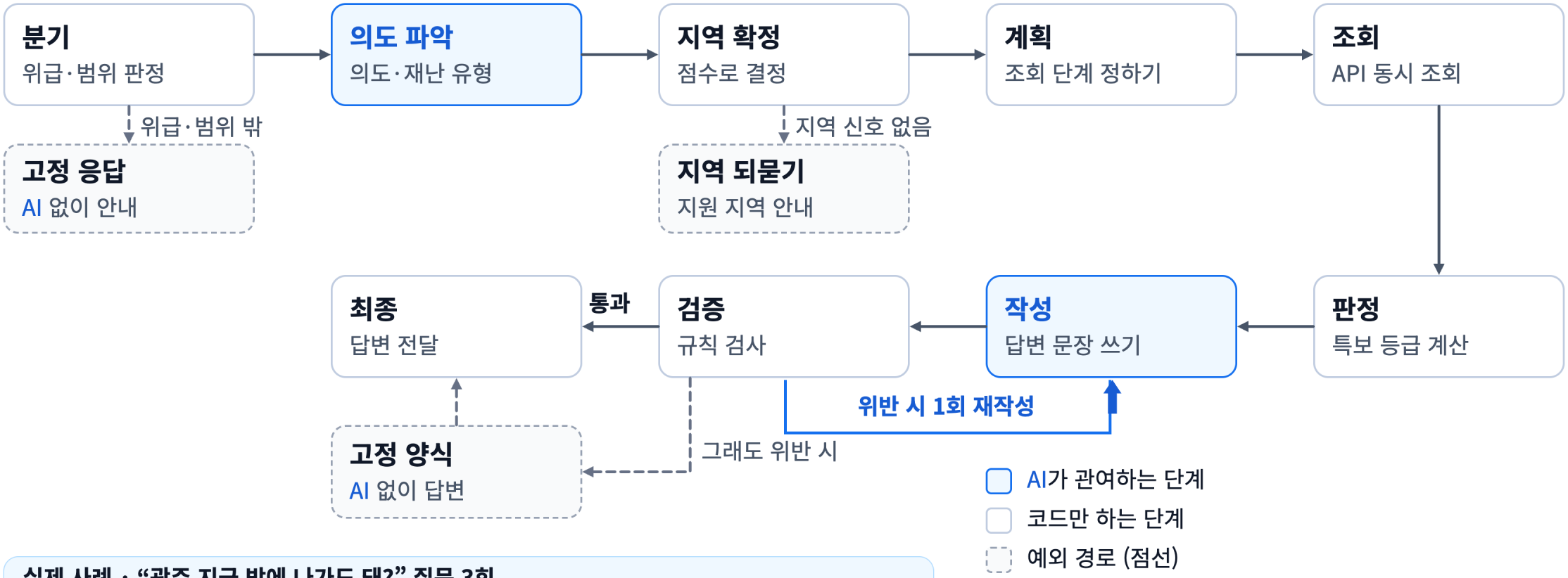
출처: 전북일보 2024.10.9 (행정안전부 자료)

"'안전한 곳으로 대피' 등 다소 모호한 내용이 많고, 과다 발송과 고령층 미수신 등의 문제가 있다"

행정안전부가 '언제, 어디로, 어떻게'를 알려 주는 표준문안을 만든다.

출처: 경향신문 2025.8.27

AI는 이해와 작성만, 계획·판단·검증은 코드가



실제 사례 · “광주 지금 밖에 나가도 돼?” 질문 3회
AI가 모두 질문에 없는 ‘태풍’을 냈고, 코드가 전부 걸러 냈다.

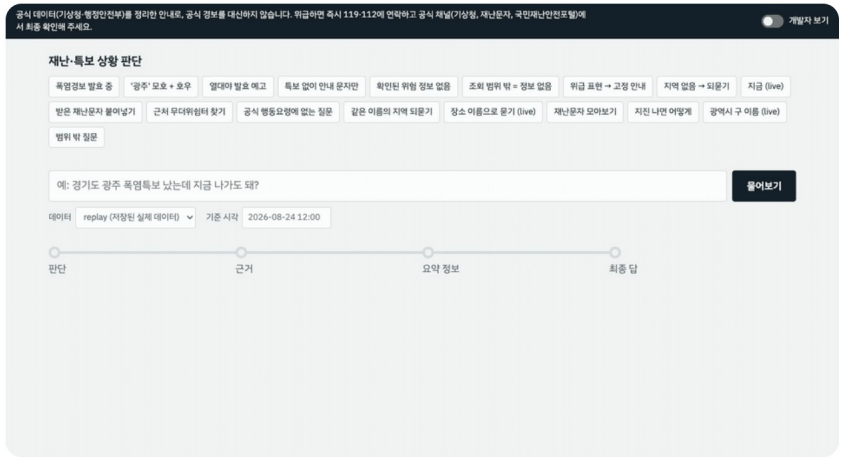
*각 단계는 LangGraph로 연결

설계 · 기술 선택

필요한 곳에, 이유가 있는 기술만 썼다

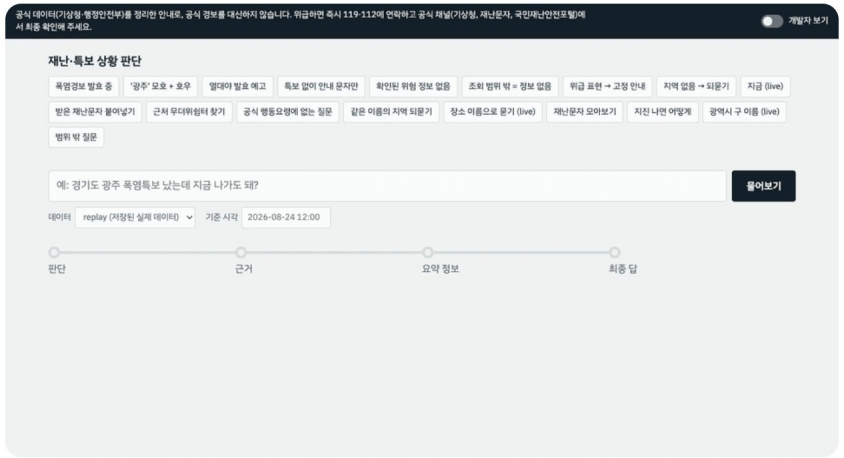
기술	생긴 문제	그래서 이렇게 썼다
Multi-Agent 구조	AI 혼자 답하면 틀려도 걸러 낼 곳이 없다.	역할이 나뉜 단계를 LangGraph로 연결했다. 판단·검증은 코드가 맡고, 틀린 답은 1회 다시 쓴 뒤 고정 양식으로 답한다.
MCP	API 8개(공공 7종 + 지도)가 키 8개로 나뉘고 형식도 제각각이다.	하나의 도구 형식으로 묶어 같은 방식으로 동시에 조회한다. API가 바뀌어도 서버 안에서 고친다.
의존성 주입	실제 모델로 테스트하면 느리고(질문 하나에 수십 초) 결과도 매번 다르다.	모델을 테스트 응답으로 바꿔 끼워 코드가 맞는지 즉시 테스트한다. 실제 모델 평가는 따로 돌린다.
LangSmith	답이 틀렸을 때 어느 단계 탓인지 알아야 고칠 수 있다.	단계별 입력과 출력을 기록해 추적하고, 평가 질문 세트도 같은 방식으로 돌린다.
모듈화 · 테스트	고칠 때마다 예전에 되던 것이 깨질 수 있다.	데이터 서버·백엔드·화면을 나누고, 단위 테스트와 평가 질문 세트를 자동으로 돌린다.
RAG는 쓰지 않음	행동요령을 문서 검색 AI로 찾을지 검토했다.	약 1,500건이고 종류·시기·대상별로 정리돼 있어, 규칙으로 고르면 같은 질문에 같은 근거가 나온다.

알려 줄 땐 근거와 함께, 없으면 없다



① “광주 폭염특보 났는데 지금 나가도 돼?”

판단→근거→요약 정보→최종 답 순으로 채워지고, 특보·행동요령·쉼터에 근거 번호가 붙는다.



② “호우 때 지하차도에서 차가 물에 잠기면 어떻게 탈출해?”

조회했지만 행동요령이 없어, 지어내지 않고 “정보 없음”으로 답한다.

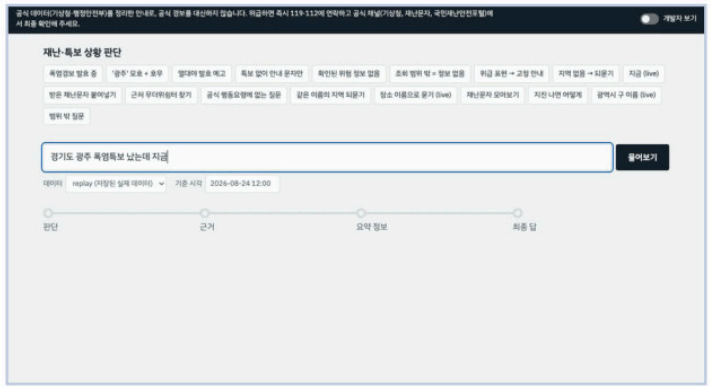
※ 실제 시연을 빨리 감은 영상. 최종 답까지의 실제 시간은 평균 약 **30초**(8번).

시연·결과 · 시연 (정지 프레임)

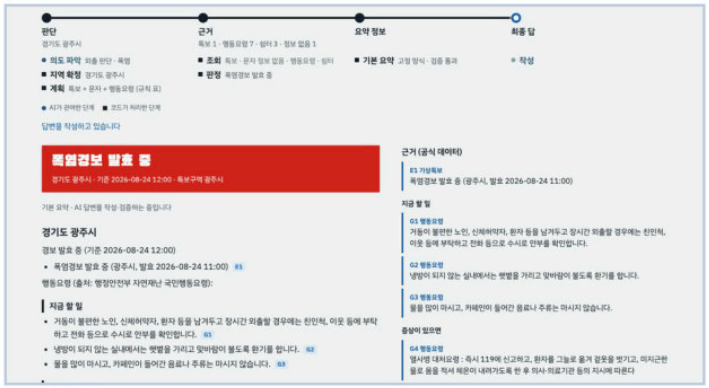
시연 영상 장면: 판단 → 근거 → 최종 답

PDF에서는 GIF가 움직이지 않아 시간 순서대로 장면을 나열했습니다. 움직이는 영상은 PPTX 슬라이드쇼에서 재생됩니다.

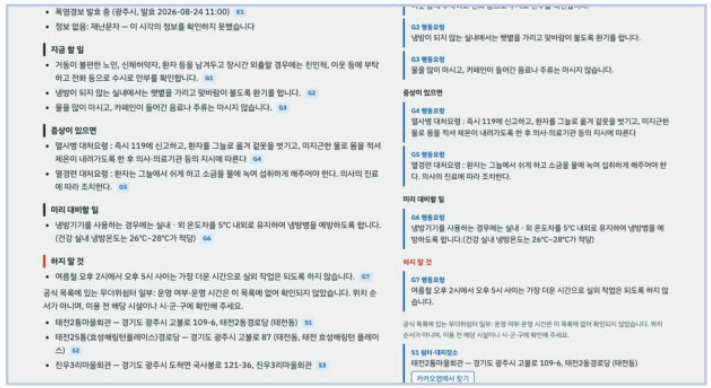
① “광주 폭염특보 났는데 지금 나가도 돼?”



1단계 · 영상 0.6초

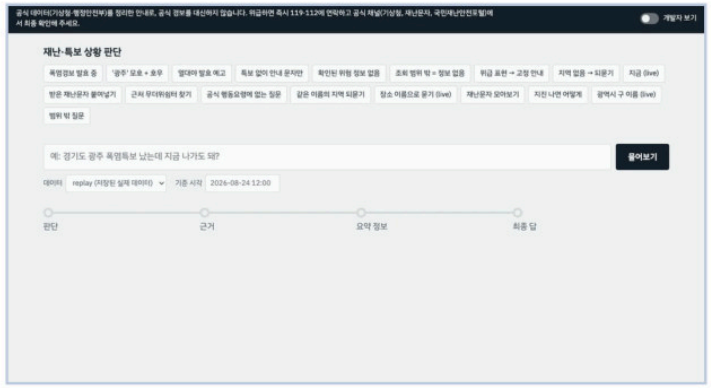


2단계 · 영상 2.5초



3단계 · 영상 4.6초

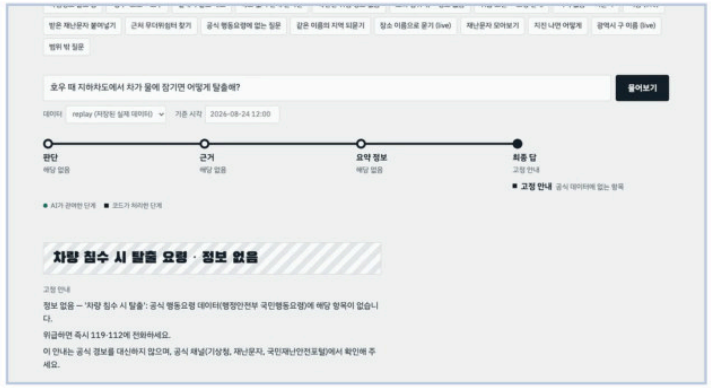
② “호우 때 지하차도에서 차가 물에 잠기면 어떻게 탈출해?”



1단계 · 영상 0.6초



2단계 · 영상 2.5초



3단계 · 영상 4.6초

그래서 이 세 가지 문제를 해결했다

문자엔 행동이 없다

지금 할 일

특보 수준에 맞춰 바로 할 행동을 먼저 보여 준다.

정보가 흩어져 있다

공식 근거

모든 안내에 특보·문자·행동요령·대피소 출처 번호가 붙는다.

AI는 지어낼 수 있다

정보 없음

공식 데이터에 없는 내용은 지어내지 않는다.

7종
연동한 공공 API (특보·문자·행동요령 4·대피소)

301곳
특보를 구분하는 전국 구역

약 1,500건
근거로 쓰는 공식 행동요령

고치기 전과 후를 같은 기준으로 쟀다

지표	고치기 전	고친 후	뜻
중복 문장 비율	12.9%	**0%**	고른 행동요령 중 거의 같은 문장 비율
실패 원인 구분	1가지	**5가지**	API 장애 5종이 보이는 서로 다른 문구 수
첫 답이 보이기까지	약 15초	**1.7초**	질문 후 검증된 기본 요약이 뜨는 시간 (평균)
최종 답까지	약 15초	**약 30초**	검증을 마친 최종 답이 나오는 시간 (평균)
평가 통과	5개 지표 만점	**7개 모두 만점**	같은 질문 44건, 채점 가능한 7개 지표
자동 테스트	108개	**186개**	고칠 때마다 돌리는 테스트

측정 조건: 같은 질문 기준. 시간은 qwen3:8b, 나머지는 테스트 모델

정확성을 속도보다 우선했다

한계	현황 (수치)	선택과 이유
공식 데이터에 없는 질문	차량 침수 탈출 요령 등이 행동요령 API 4종에 없음	지어내지 않고 “정보 없음”으로 답함
최종 답이 느리다	평균 약 30초 (고치기 전 약 15초)	PC에서 모델을 돌려 느리다. 대신 근거부터 먼저 보여 준다
검증 후에야 최종 답	첫 답 1.7초, 최종 답 12~47초	틀린 안내보다 늦은 안내가 낫다
API 호출 한도	API별 하루 1,000건, 현재 수십 건	지금은 충분. 확대하면 캐시 필요
낯선 표현은 유형을 못 잡음	“엄청 더운데”처럼 화이트리스트에 없는 질문은 놓칠 수 있음	지어내지 않으려고 글자 근거가 있는 유형만 인정함

측정 조건: LOCAL의 8B 모델(qwen3:8b), 같은 질문 7개 × 3회

한계·개선 · QA로 고친 것

QA를 통해 4가지를 고쳤다

답변 품질 QA

중복 문장, 섞인 문장, 개발자 말투를 잡고 코드로 검증한다.

측정: 중복 문장 비율

실패 원인 분류

"정보 없음"과 "조회 실패"를 구분하고, 실패는 원인별로 나눠 알려준다.

측정: 실패 원인 구분

단계별 표시

판단 → 근거 → 요약 정보 → 최종 답 순서로, 확인된 것부터 먼저 보여 준다.

측정: 첫 답이 보이기까지 · 최종 답까지

평가·검증 체계

단위 테스트와 평가 질문 세트로, 고칠 때마다 예전에 되던 것이 깨지지 않았는지 확인한다.

측정: 평가 통과 · 자동 테스트

한계·개선 · 다음 계획

다음에 할 일 4가지

전국 특보 지도

지금은 지역을 입력해야 한다. 특보가 난 지역을 지도에 색으로 보여 준다.

대피소 길찾기

쉼터·대피소 안내에 카카오맵 길찾기 링크를 붙여, 바로 이동할 수 있게 한다.

문장 단위 검증

지금은 답 전체를 검증한 뒤 보여 준다. 문장마다 검증하며 바로 보여 대기 시간을 줄인다.

이름 · 표현 이해 확대

이름이 겹치는 지역(광주, 고성군)과 "엄청 더운데" 같은 말을 놓치지 않게, AI가 근거 문구를 인용하면 코드가 확인한다.

한 줄 요약

지금 할 일을 공식 근거로 알려 주고, 모르면 모른다고 말한다.

재난문자 모아보기 · 질문 받겠습니다

감사합니다

제출 가이드 항목별 상세 자료

부록

13 프로젝트 소개와 주요 Task

14 Multi-Agent 구성

15 나머지 기술 요소 상세

16 LangSmith 추적 화면

17 구현 화면과 최종 실행 화면

18 개선 전/후 측정 방법

19 개선 전/후 비교 결과 화면

20 느낀 점 및 향후 개선

부록 · 진행 사항

프로젝트 TASK 소개

재난 특보·문자·행동요령을 공식 근거로 묶어, 지금 할 일을 알려 주는 AI 에이전트

주요 Task	한 일	결과
공공데이터 연동 (MCP)	특보·문자·행동요령 4종·대피소(공공 API 7종)와 지도 API를 MCP 서버 하나로 묶었다.	키 8개·발급처 3곳을 같은 방식으로 조회
답변 파이프라인	의도 파악 → 지역 확정 → 계획 → 조회 → 판정 → 작성 → 검증을 LangGraph로 연결했다.	판단·검증은 코드, AI는 의도 파악·작성
안전장치	검증 위반 시 1회 다시 쓰고, 그래도 위반이면 고정 양식으로 답한다. 근거가 없으면 "정보 없음".	검증되지 않은 문장은 사용자에게 가지 않음
단계별 화면	판단 → 근거 → 요약 정보 → 최종 답 순서로 진행을 보여 주고, 판정 띠와 근거 번호를 붙인다.	확인된 것부터 먼저 보여 줌
평가·테스트	단위 테스트, 평가 질문 세트, 가짜 모델·실제 모델 비교 실행.	테스트 193개, 평가 58/58 통과
실행 환경	도커 컨테이너 3개(데이터 서버·백엔드·화면), Local PC의 qwen3:8b 모델.	학습용(비상업), 로컬 실행

Multi-Agent 구성: 단계마다 AI인지 코드인지

단계	하는 일	AI인지 코드인지
분기	위급 표현·범위 밖 질문을 고정 응답으로 끝냄	코드 (AI 없이 처리)
의도 파악	질문의 의도와 재난 유형을 이해	AI + 규칙. AI가 낸 유형은 질문 글자에 근거가 있을 때만 채택
지역 확정	지역 이름을 점수로 정하고, 겹치면 되묻기	코드
계획	어떤 자료를 조회할지 규칙 표로 결정	코드
조회	공공 API 동시 조회 (MCP)	코드
판정	특보 등급 계산	코드
작성	근거 번호를 붙여 답변 문장 작성	**AI**
검증	인용·특보 누락·날것 태그 등을 규칙으로 검사	코드
최종 · 재작성 · 고정 양식	통과하면 전달, 위반하면 1회 재작성, 그래도 위반이면 고정 양식	코드

AI(LLM)를 호출하는 곳은 의도 파악과 작성 두 곳뿐이고, 나머지 단계는 코드가 맡는다.

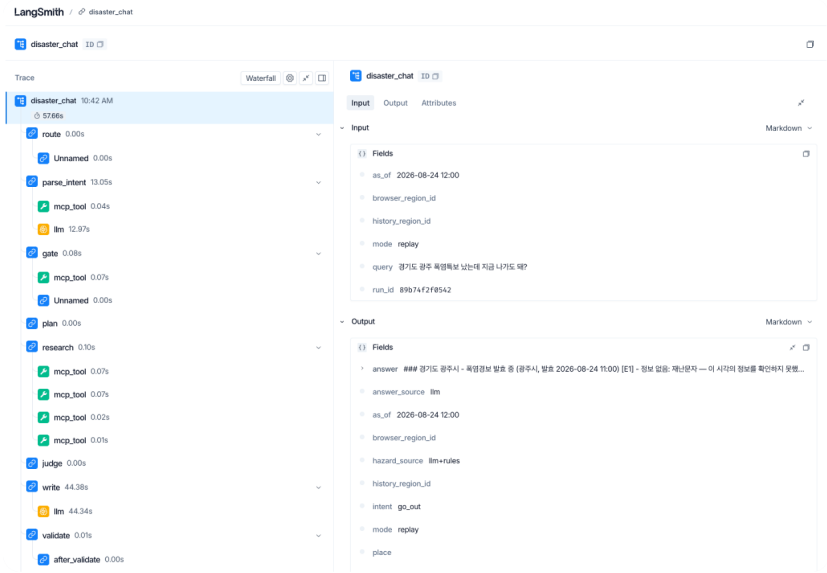
부록 · 기술 요소

나머지 기술 요소 상세

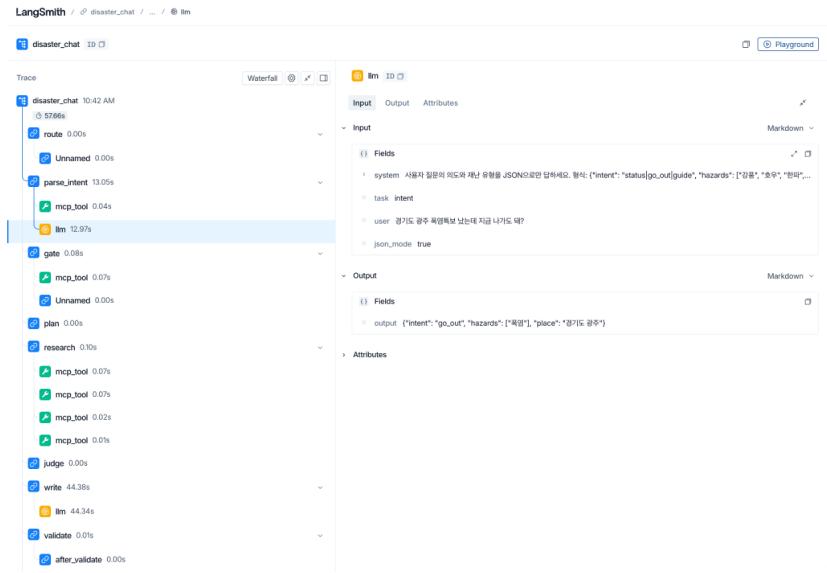
기술	적용	근거
MCP	공공 API 7종+지도 API를 MCP 서버(도구)로 제공하고, 백엔드는 같은 방식으로 호출한다.	키 8개·발급처 3곳을 서버 안에 숨김
의존성 주입	모델(LLM_BACKEND: 테스트/실제)과 데이터(DATA_MODE: 저장본/실시간)를 설정으로 교체한다.	같은 평가 세트를 테스트 모델과 qwen3:8b로 각각 통과
LangSmith	단계별 입력·출력을 추적하고, 평가 질문 세트를 같은 방식으로 실행한다.	평가 질문 58건 (최신), 추적 화면은 다음 쪽
모듈화	데이터 서버 · 백엔드 · 화면을 컨테이너 3개로 나눴다.	포트 8200 / 8100 / 8080
단위 테스트	MCP 서버와 백엔드를 각각 테스트하고, 평가 질문 세트로 회귀를 확인한다.	MCP 82(스킵 2) + 백엔드 111 = 193개 통과
RAG	검토 후 쓰지 않음. 행동요령 약 1,500건이 종류·시기·대상별로 정리돼 있어 규칙으로 고른다.	같은 질문에 같은 근거를 위함

부록 · 기술 요소

LangSmith 추적 화면



질문 한 번(총 57.7초)의 전체 추적. 단계별 시간과 MCP 도구 호출이 보이고, AI 호출은 의도 파악(13.1초)과 작성(44.4초) 두 곳이다.



의도 파악 AI 호출의 입력(질문)과 출력(의도 go_out, 재난 유형 폭염, 장소 경기도 광주).

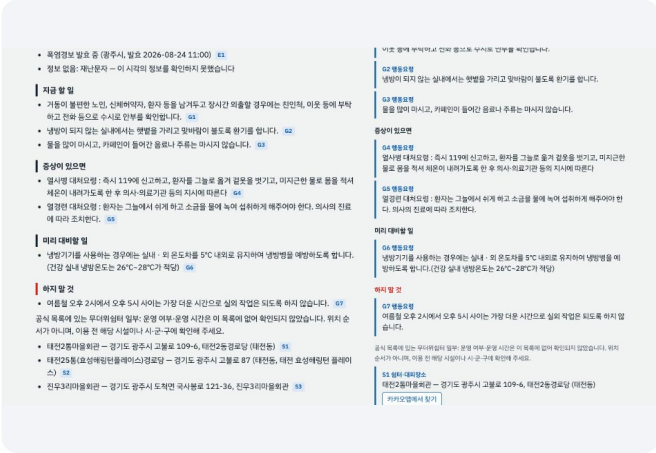
부록 · 구현 화면

구현 화면과 최종 실행 화면

질문 1·2: “경기도 광주 폭염특보 났는데 지금 나가도 돼?” (기준 2026-08-24 12:00) · 질문 3: “호우 때 지하차도에서 차가 물에 잠기면 어떻게 탈출해?”



진행 중: 노선도와 판정 띠(폭염경보 발효 중). 기본 요약은 검증을 통과했고 AI 답변을 작성하는 중.



최종 답: 행동요령과 쉼터에 근거 번호(E1, G1~G7, S1~S3)가 붙고, 운영 여부를 모르는 쉼터는 그렇게 밝힌다.



공식 데이터에 없는 질문(차량 침수 탈출)은 지어내지 않고 “정보 없음”과 119·112 안내만 보여 준다.

부록 · 개선 전/후 비교

개선 전/후 측정 방법

지표	어떻게 쟀나	쓴 모델
중복 문장 비율	공통 질문 16건에서 코드가 고른 행동요령 문장 중 거의 같은 문장의 비율	테스트 모델
실패 원인 구분	API 장애 5종을 넣었을 때 보이는 서로 다른 안내 문구 수	테스트 모델
첫 답이 보이기까지	질문 후 검증된 기본 요약이 뜨는 시간, 평균	qwen3:8b
최종 답까지	검증을 마친 최종 답이 나오는 시간, 질문 7개 × 3회, 질문별 중앙값의 평균	qwen3:8b
평가 통과	같은 질문 44건, 채점 가능한 7개 지표	가짜 모델
자동 테스트	고칠 때마다 돌리는 테스트 수	모델 없음

같은 질문 세트로 고치기 전과 고친 후를 비교했다. 응답 시간만 최신 코드로 다시 쟀다. 수치는 발표 7쪽, 실행 화면은 다음 쪽에 있다.
재시도 횟수: 최신 평가에서 AI 작성 호출 49회 중 재작성은 6회였다(고친 후 기준).

부록 · 개선 전/후 비교

개선 전/후 비교 결과 화면

eval/compare_before_after.py — 중복 문장(dup)·자동 테스트(tests) · Before aec2694 / After a40cf11

```
$ git log -1 --format='%h %ad %s' --date=short aec2694   # Before (구 해시 e13c10f, 같은 트리)
aec2694 2026-10-20 docs: T-027 집중 강화와 qwen 재평가 결과
$ git log -1 --format='%h %ad %s' --date=short a40cf11   # After  (구 해시 6c62b95, 같은 트리)
a40cf11 2026-10-21 docs+fix: 공개 문서 정리와 qwen 전체 평가 기록, 광역시 구 '시 전체 기준' 문장 검증 (N, T-035)

$ .venv/bin/python eval/compare_before_after.py dup   --tree ../baseline_work/aec2694 --tree ../baseline_work/a40cf11 --out results11/dup.json
$ .venv/bin/python eval/compare_before_after.py tests --tree ../baseline_work/aec2694 --tree ../baseline_work/a40cf11 --out results11/tests.json

$ jq 'map_values(del(.detail))' dup.json
{
  "aec2694": {
    "questions": 20,
    "answers_with_guides": 16,
    "guide_sentences": 62,
    "dup_sentences": 8,
    "dup_sentence_ratio": 0.129,
    "answers_with_dup": 4,
    "answer_dup_ratio": 0.25,
    "threshold": 0.4
  },
  "a40cf11": {
    "questions": 20,
    "answers_with_guides": 20,
    "guide_sentences": 140,
    "dup_sentences": 0,
    "dup_sentence_ratio": 0.0,
    "answers_with_dup": 0,
    "answer_dup_ratio": 0.0,
    "threshold": 0.4
  }
}

$ jq -r 'to_entries[] | .key as $c | .value.detail[] | select(.dup_pairs>0) | "\($c)  \(.q)  guides=\(.guides) dup_pairs=\(.dup_pairs)"' dup.json
aec2694  경기도 광주 폭염 행동요령 알려줘  guides=4 dup_pairs=1
aec2694  경기도 광주 폭염 때 어떻게 해야 해?  guides=4 dup_pairs=1
aec2694  경기도 광주 호우 행동요령 알려줘  guides=6 dup_pairs=1
aec2694  경기도 광주 호우 때 어떻게 해야 해?  guides=6 dup_pairs=1

$ jq -c '.[]' tests.json | paste -d' ' <(jq -r 'keys_unsorted[]' tests.json) -
aec2694 {"mcp":43,"backend":65,"skipped":2,"ok":true,"total":108}
a40cf11 {"mcp":81,"backend":105,"skipped":2,"ok":true,"total":186}
```

중복 문장 · 자동 테스트 비교 출력

지표 3 — 응답 시간 (Before/After)

출처: doc/internal/측정_결과.md — '표'의 지표 3행과 '지표 3 상세' 표를 그대로 옮김 (compare_before_after.py timing 실행 결과, 실행 2026-10-21 · Before `aec2694` (T-027 검증 강화 직후) · After `a40cf11` · 스크립트 `eval/compare_before_after.py`)

지표	Before `aec2694`	After `a40cf11`	비고
3. 첫 내용(판정 카드)이 보이기까지 — qwen, 질문 7개 × 3회, 질문별 중앙값의 중앙값	2.98초 (최소 2.56 · 최대 4.0)	1.66초 (최소 1.57 · 최대 2.03)	첫 이벤트는 두 커밋 모두 21회 전부 `judgement` 카드. Before는 계획 LLM 호출이 그 앞에 있다
3'. 첫 답변 글이 보이기까지	13.88초 (최소 9.64 · 최대 22.86)	1.66초 (최소 1.57 · 최대 2.03)	Before: 첫 `token` (검증 뒤라 final과 같은 시각), After: `quick` (검증 통과 템플릿)
3''. 최종 답변까지	13.88초 (최소 9.64 · 최대 22.86)	41.65초 (최소 9.81 · 최대 73.51)	**After가 느리다** — 답변이 길어짐(행동요령 묶음·증상 대처·쉼터·면책)

지표 3 상세 (qwen3:8b, replay, 각 3회)

질문	Before 첫 카드 / 첫 글 / 최종	After 첫 카드 / 첫 글 / 최종
경기도 광주 폭염특보 났는데 지금 나가도 돼?	4.0 / 19.6 / 19.6	1.66 / 1.66 / 73.51
광주 호우 어때?	2.91 / 22.86 / 22.86	1.62 / 1.62 / 42.01
경기도 광주 열대야 어때?	2.98 / 11.17 / 11.17	1.68 / 1.68 / 41.65
경기도 광주 강풍 상황 알려줘	3.3 / 17.27 / 17.27	1.57 / 1.57 / 9.81
광주광역시 날씨 위험해?	3.27 / 12.52 / 12.52	2.03 / 2.03 / 21.08
경기도 광주 폭염 어때?	2.56 / 9.64 / 9.64	1.64 / 1.64 / 21.55
광주광역시 호우주의보 행동요령 알려줘	2.91 / 13.88 / 13.88	1.69 / 1.69 / 46.75

- After: 도커(`a40cf11` 코드로 빌드한 이미지), Before: worktree에서 `uvicorn --factory server:app_factory` (포트 8110, 같은 Ollama). 같은 질문·기준 시각·모델 `LLM_NO_THINK=1`.
- 해석(추정): 최종 답변이 늘어난 것은 write 출력이 길어진 탓으로 본다(T-035 측정: 출력 토큰 중앙값 455, 초당 약 18~20토큰). 이 측정에서는 토큰 수를 따로 재지 않았다.

응답 시간 비교 표

같은 질문 세트로 고치기 전(aec2694)·후(a40cf11)를 비교 실행한 출력과 응답 시간 표. 해시는 작성자 정보를 통일하기 전 e13c10f·6c62b95와 같은 코드다.

느낀 점 및 향후 개선

느낀 점

· AI에게 맡길 일과 코드가 할 일을 나누는 것이 정확성의 핵심이었다. AI가 질문에 없는 '태풍'을 3번 지어냈을 때 코드 검증이 모두 걸러 냈다.

· 측정하니 개선이 숫자로 보였다. 대신 최종 답이 약 15초에서 약 30초로 느려진 대가도 같이 보였다.

· 공식 데이터에 없는 내용은 "정보 없음"이라고 답하는 쪽이 더 믿을 만하다고 느꼈다.

향후 개선

· 검증에서 반려된 사유를 모아 분석한다.

· 호출이 늘면 캐시를 둔다.

· 침수 피해지도, 열대야(무더위) 지도, 한파 지도 등

· 그 밖의 계획(전국 특보 지도, 대피소 길찾기, 문장 단위 검증, 지역 이름 구분)은 발표 10쪽에 있다.

· 침수, 도로통제 등의 상황 시에 안전한 길이라던가 경로를 도입하고 싶었는데, ITS(도로정보 API)가 월 호출제한 100건이라서 사용을 못 했다. 기간이 충분했다면 호출량 증량 신청한 후에 적용해보고 싶었다.